

An engineering and mathematical audit of Asymptotic 1.8.0

Correctness, precision, algorithms, Wolfram Language integration, and development priorities

Prepared for Vladimir Reshetnikov

Review date 9 September 2026

Repository <https://github.com/VladimirReshetnikov/Asymptotic>

Audited commit 07a9781212beb2eeb9ff16aa625b50ac27974078

Snapshot timestamp 2026-09-09 19:01:30 UTC

Central conclusion. The repository is a substantial precision-aware layer on top of Wolfram Language, not merely a workaround for `InverseSeries`. Its strongest contributions are explicit generalized coefficient algebras, complete-block inversion, retained branch information, and separate formal, asymptotic, numerical, and certified evidence. The most urgent work is to make those contracts consistent across shared primitives, optional representations, and refinement paths.

Evidence boundary. This review combines source inspection, mathematical derivations, and executed independent Python checks. Native Wolfram evaluation was attempted but unavailable. The supplied Wolfram regression and comparison programs are therefore *reproduction artifacts, not claimed test results*. The proposed source patch has not been validated in a Wolfram kernel.

Article, runnable independent checks, native test proposals, a source-verified patch generator, and machine-readable evidence are supplied together. No repository files were modified remotely.

Executive assessment

The right next step for Asymptotic is *consolidation before another large increase in function coverage*. Version 1.8.0 already combines ordinary generalized power–log inversion, multiple inversion algorithms, logarithmic coordinates, exact inverse cores, flat exponential sectors, finite Fourier coefficients, native special-function import, Gamma and Barnes G inversion, arithmetic on series objects, numerical checks, and exact interval certificates. Its guide explicitly distinguishes many of these scales and their cutoff conventions. A review based only on the original inverse-function question would substantially understate the current implementation.[1, 2, 3]

The audit identifies two especially important shared-infrastructure problems. First, building an optional `SeriesData` property can allocate a dense list whose length is essentially unrelated to the number of nonzero generalized terms. Second, a noninteger power applied inside a compound observable can take a path that lacks the real-branch protection present in the direct `SeriesPower` interface. There is also a separate refinement defect: an operation can return a conservative expansion far less accurate than the requested refined cutoff, without marking the request unsuccessful. These are different failure modes and should not be grouped together as “incorrect coefficients.”[3, 4]

The performance issue with the clearest exact witness is structural. For ordinary inverse construction, choosing Newton or grouped Lagrange does not eliminate the wrapper’s prior multi-index enumeration. A 30-block polynomial example has 23,025 retained multi-indices even before its frontier is included. An independent exact counting program verifies that number. The proposed optimization must remove the enumeration requirement from the whole constructor, not merely accelerate its inner Newton routine.[3, 7]

ID	Priority	Finding and immediate consequence
F01	High	Eager dense export can exhaust memory for a tiny sparse result.
F02	High	Nested fractional observables can bypass a required real-branch check.
F03	High	Newton/grouped construction still pays combinatorial Lagrange enumeration costs.
F04	High	Derived refinement can silently miss its requested precision by many powers.
F05	Low	Residual normalization text omits the limiting target offset, although the computation subtracts it.
F06	Medium	Logarithm canonicalization can invoke unrestricted integer factorization outside the advertised resource budget.
F07	Medium	Flat-sector rate recognition rejects commensurate rates such as 2,3 because it requires integer multiples of the smallest supplied rate.

Priority ranks engineering attention, not a standardized vulnerability score. F07 is a documented restriction, not a wrong answer on admitted input. F02 and F04 remain source-traced rather than kernel-observed; native reproductions are supplied.

The package should be positioned as a *structured, inspectable asymptotic computation system that interoperates with the native engine*. Wolfram already supplies much broader asymptotic functionality than ordinary Taylor series. Conversely, a native expression containing an asymptotic approximation is not automatically a replacement for the package’s recorded coefficient algebra, source branch, replayable computation state, or explicit interval certificate. The useful comparison is therefore capability by capability, rather than “the package can do asymptotics and Mathematica cannot.”[27, 29, 30]

Contents

Executive assessment	1
1 Scope, snapshot, and evidential method	4
1.1 What was reviewed	4
1.2 Evidence categories	4
1.3 Runtime access and reproducibility	5
2 Architecture and the contracts worth preserving	5
2.1 The ordinary mathematical model	5
2.2 Complete blocks, not individual printed terms	5
2.3 Jet precision as mathematical data	6
2.4 Multiple coefficient algebras and result families	7
3 Mathematical checks of the main mechanisms	7
3.1 The generalized Lagrange coefficient formula	7
3.2 Why finite exponent truncation is well defined	8
3.3 Transport of an unknown forward remainder	8
3.4 Newton precision and independent residual checks	9
3.5 Gamma inversion around an exact Lambert core	9
3.6 Barnes G inversion and a different logarithmic coefficient variable	9
4 Comparison with built-in Wolfram Language	10
4.1 The comparison boundary	10
4.2 Capability matrix	10
4.3 Where the package is genuinely differentiated	11
4.4 Where native capabilities should be reused rather than recreated	12
5 Concrete findings and proposed corrections	12
5.1 F01: An optional dense export can dominate the entire computation	12
5.2 F02: A nested fractional observable can bypass the real-branch guard	13
5.3 F03: The constructor defeats part of the Newton and grouped-Lagrange advantage	14
5.4 F04: Derived refinement can return a much weaker expansion than requested	15
5.5 F05: Residual normalization text loses the target offset	16
5.6 F06: Exact logarithm canonicalization can trigger unbounded factorization	17
5.7 F07: The flat-rate test is narrower than commensurability	17
6 Performance and resource engineering	18
6.1 Separate the relevant measures of work	18
6.2 Avoid expression growth before checking the budget	19
6.3 Weight queues and collision-aware support	19
6.4 Retained state is useful, but its cost should be visible	20
6.5 Interpreting the repository's existing benchmarks	20
7 Specialized scales: strengths, gaps, and extension boundaries	20
7.1 Native special-function import is an adapter, not an independent oracle	20
7.2 Gamma and Barnes operations need a reusable coefficient-ring abstraction	21
7.3 Fourier coefficients: exact closure within a deliberately finite class	21
7.4 Flat sectors: preserve the exact zero sector	22
7.5 Logarithmic coordinates and special-function adapters	22
8 Certificates and numerical verification	23
8.1 What the exact certificate actually proves	23
8.2 What it does not prove	23
8.3 Numerical checks are useful but should not borrow certificate vocabulary	23
8.4 A credible route to special-function certification	24
9 API and user experience	24
9.1 Make a precision request a typed object	24

9.2	Separate absolute observables from displacement powers	25
9.3	Assumptions should follow one predictable rule	25
9.4	Preserve unknown, unsupported, and disproved as different outcomes	26
9.5	Expose capabilities and a schema version	26
9.6	Display, evaluation, and provenance	26
10	Testing, packaging, and release discipline	27
10.1	The existing validation is substantial but scoped	27
10.2	CI should execute semantics as well as bundle freshness	27
10.3	Tests that target semantic invariants	27
10.4	Reproducible single-file distribution	28
11	A concrete improvement architecture	28
11.1	One shared semantic kernel, several coefficient algebras	28
11.2	Backward precision propagation	29
11.3	Independent evidence objects	29
11.4	A strategy planner with observable decisions	30
12	Development priorities and acceptance milestones	30
12.1	Milestone A: repair and make failure honest	30
12.2	Milestone B: remove method-independent combinatorial bottlenecks	30
12.3	Milestone C: stabilize the public model	30
12.4	Milestone D: extend the most economical mathematical cases	31
12.5	Milestone E: expand rigorous numerical evidence	31
12.6	Longer-range work that needs a new theory boundary	31
13	Executed independent checks and their interpretation	31
13.1	Exact checks	31
13.2	Irrational-power inverse check	32
13.3	Gamma and Barnes inverse check	32
13.4	Native results still to obtain	33
14	Conclusion	33
A	Artifact guide and reproduction commands	33
B	Module coverage and architectural map	34
C	Suggested regression families beyond the included cases	36
	References and pinned source locations	37

1 Scope, snapshot, and evidential method

1.1 What was reviewed

The review is tied to commit `07a9781212beb2eeb9ff16aa625b50ac27974078`, whose package version is 1.8.0. The guide requires Wolfram Language 15.0 or later. The canonical kernel implementation comprises 38 `.wl` files, including the entry file; a generated repository-root `AsymptoticInverse.wl` bundles those modules for single-file loading. This article concerns that snapshot, not a moving copy of `main`.^[1, 2, 25]

The most intensive inspection covered the ordinary jet algebra and inverse constructor, explicit series operations, incremental inversion and refinement state, numerical checks, exact certificates, the native special-function importer, and the main flat, Fourier, Gamma, and Barnes inverse mechanisms. Public documentation, the standalone workflow, focused test runner, and selected mathematical article sections were also inspected. The module coverage table in Appendix B distinguishes complete or substantial source review from interface-level review. It is not a claim that every branch in every auxiliary adapter was audited.

The repository contains historical research material and many test files. Those are useful context, but this review did not formally verify all historical reports, re-prove every theorem in the full repository article, or execute the full upstream suite. In particular, a test runner’s existence and a checked-in result file are different kinds of evidence from executing those tests against the exact source under review.

1.2 Evidence categories

Source-confirmed fact means a property directly visible in the pinned implementation: an allocation formula, an option default, a dispatch branch, a required input condition, or an explicit validation rule. For example, the dense coefficient-list length in F01 can be determined exactly from source without allocating the list.

Source-traced behavioral finding means an execution path has been followed far enough to identify an apparent contract violation, and a minimal native reproduction is supplied, but the Wolfram kernel result was not observed during this review. F02 and F04 are reported this way. Their mathematical counterexamples are independent of the unavailable runtime, but exact message text, intermediate evaluation, and final expression formatting still deserve native confirmation.

Independent mathematical check means a separate Python/SymPy or mpmath calculation was actually executed. It checks a formula, exact count, or numerical consequence. It is neither a translation of the complete Wolfram package nor a substitute for testing Wolfram evaluation semantics. The supplied ledger records 22 such checks, all passing.

Repository-reported validation means the repository itself records a result. The current validation documentation is notably explicit about focused versus complete coverage. Such results are attributed rather than rebranded as this audit’s execution.^[22]

Recommendation means a proposed change, including a new API, representation invariant, performance design, or future feature. Proposed code is labeled separately from implemented upstream behavior. The three surgical source edits supplied here are not a general corrected distribution of the repository.

1.3 Runtime access and reproducibility

A Wolfram connector was used in an attempt to obtain a native evaluation environment; the evaluator service failed. There was no usable local Wolfram kernel either. Accordingly, no native head-to-head timing, no execution of the proposed `.wlt` tests, and no claim of a fully passing patched package is made.

The independent checks were executed with Python 3.13.5, SymPy 1.14.0, and mpmath 1.3.0. Their recorded run completed 22 checks in 10.7907 seconds in the audit environment. Eleven additional tests of the audit's own patch/planning tooling also passed. These timings characterize the small independent checks, not Asymptotic's performance. Exact outputs, numerical values, environment versions, and test names are retained in `results/oracles.json` and `results/tool-tests.txt`.

For later reproduction, both modular and standalone source versions should be recorded. A checkout at a commit is a starting point, but a source modification after checkout changes the tested artifact. The supplied native runner therefore records hashes of the loaded modules and verifies that they did not change during execution. The source patch generator additionally checks the canonical entry file's Git blob identity before accepting any replacement anchors.

2 Architecture and the contracts worth preserving

2.1 The ordinary mathematical model

The core model is a finite real germ on a positive local coordinate $u \rightarrow 0^+$:

$$f(u) = y_0 + au^p \left(1 + \sum_{i=1}^m u^{\delta_i} B_i(\log u) \right), \quad p \neq 0, \quad a \neq 0, \quad \delta_i > 0, \quad (1)$$

where each B_i is a polynomial. The leading coefficient must have a proved sign; the local coordinate records the chosen side of a finite endpoint or a real infinity. The implementation supports exact real numerical exponents in ordered exponent mode and a distinct depth mode for suitable symbolic exponents. It does not simply round irrational exponents onto a numerical grid.[3, 2]

At a finite endpoint, $u = x - x_0$ or $u = x_0 - x$ according to the selected side. At positive infinity, $u = 1/x$; at negative infinity, $u = -1/x$. These substitutions make the small coordinate positive even when the original variable or its displacement is negative. Positivity is part of the branch semantics, not a display convention. A correct result must retain enough information to distinguish the source approach from the target approach.

The uniformizer is

$$z = \left(\frac{y - y_0}{a} \right)^{1/p} > 0, \quad L = \log z, \quad (2)$$

with the corresponding interpretation at an infinite target limit. Expressions such as fractional powers and logarithms are then assembled on this specified real branch. The absolute remainder coordinate can be a signed target displacement or a reciprocal target, so reading `s["RemainderVariable"]` is safer than assuming it is simply y .

2.2 Complete blocks, not individual printed terms

A power-log block is $u^\alpha P(\log u)$ with its *entire* polynomial P . A cutoff retains complete blocks at exponents strictly below the requested bound. This grouping is important twice: logarithmic

terms at a common exponent must remain together, and different multi-indices can generate exactly the same exponent and cancel.[3, 20]

For instance, the inverse of $x + x^2 + x^3$ begins

$$g(y) = y - y^2 + y^3 + 0y^4 - 4y^5 + 14y^6 + \cdots. \quad (3)$$

The missing fourth-power block is not evidence that each contributing Lagrange coefficient is zero. It is a collision between different index contributions. Counting indices rather than complete nonzero weights produces a misleading term goal; choosing one index as the omitted frontier can produce a wrong remainder coefficient. The existing complete-weight aggregation is a strength that optimizations must preserve.

Depth truncation is deliberately different. With more than one gap, retaining total perturbation degree at most N need not retain an initial segment of the ordered asymptotic scale. The repository article already explains this distinction. It is therefore appropriate to offer depth truncation, but inappropriate to present its order as if it were an ordinary exclusive exponent cutoff.[20]

2.3 Jet precision as mathematical data

The ordinary internal representation is essentially

$$(T, P, D), \quad T = \{(\alpha_j, Q_j)\}, \quad f(u) = \sum_j u^{\alpha_j} Q_j(\log u) + \mathcal{O}(u^P \mathcal{M}(u)^D), \quad (4)$$

where $\mathcal{M}(u) = 1 + |\log u|$ and $P = \infty$ denotes an exact finite expression. An empty list T with finite P is *not exact zero*; it is a pure remainder. This distinction is central to F02 and to safe cancellation in arithmetic.[3, 4, 6]

The public `GeneralizedSeries` object stores a finite expression together with remainder, scale, coordinate, branch, and provenance information. Different families also retain models, coefficient variables, inverse cores, source recipes, or computation state. The generic ordered representation used by explicit operations has the form

$$\text{Offset} + \text{Prefactor} (\text{Jet} + \text{remainder}). \quad (5)$$

The prefactor is exact. Its growth or decay can make an error small relative to the prefactor but large in absolute magnitude. Consequently, a numeric cutoff has no meaning without its coordinate and normalization.

Invariant 1 (Precision must not be created by representation changes). Converting, displaying, flattening, composing, or refining a series may weaken a justified remainder. It may strengthen that remainder only using additional source evidence, a proved cancellation with appropriate dependency information, or a verified new coefficient computation.

This invariant is already reflected in several parts of the repository. Unknown errors in composite arithmetic are kept separately. Differentiation requires a derivative contract. Refinement checks retained state and model compatibility. Native import does not merely discard a native order term. The main engineering opportunity is to make this invariant universal rather than route-dependent.[4, 8, 13, 6]

2.4 Multiple coefficient algebras and result families

The repository has moved beyond one coefficient ring. The ordinary core uses polynomial logarithms. Fourier coefficients use finite sums of logarithmic polynomials times complex exponential modes, together with real-conjugacy conditions. Gamma and Barnes inverses retain complete polynomials in an inverse logarithm of an exact core. Flat expansions have an exponential-sector degree separate from their coefficient orders. Native special-function output may require several exact carriers and bounded oscillatory coefficients.[17, 15, 16, 18, 13]

That diversity is mathematically justified, but a single association-backed result head does not by itself provide a uniform algebra. The public head should therefore be thought of as a tagged family of representations. Operations need explicit capability checks: being a `GeneralizedSeries` does not imply that the object admits one ordered power-log cutoff, a `SeriesData` conversion, ordinary composition, or arbitrary differentiation.

3 Mathematical checks of the main mechanisms

3.1 The generalized Lagrange coefficient formula

For the model in (1), write $g(y) = z(1 + U)$ in the positive source coordinate. For a fixed observable power r , the coefficient formula used by the ordinary Lagrange route is

$$g(y)^r = z^r \sum_{\mathbf{k} \in \mathbb{N}^m} z^{\mathbf{k} \cdot \boldsymbol{\delta}} C_{\mathbf{k}}^{(r)}(L), \quad C_{\mathbf{0}}^{(r)} = 1. \quad (6)$$

Put $n = |\mathbf{k}|$ and $\sigma = \mathbf{k} \cdot \boldsymbol{\delta}$. For $n \geq 1$,

$$C_{\mathbf{k}}^{(r)}(L) = \frac{(-1)^{nr}}{p^n \prod_i k_i!} \left[\prod_{j=1}^{n-1} \left(\frac{d}{dL} + r + \sigma + pj \right) \right] \prod_i B_i(L)^{k_i}. \quad (7)$$

The empty operator product at $n = 1$ is the identity. This is an efficient formulation because each Euler-type differential operator preserves the polynomial coefficient algebra. It avoids repeatedly differentiating a fully expanded expression in the physical variable. The implementation has independent uncached, cached, and grouped forms of this calculation.[3, 7]

The source does not obtain exactness by numerical cancellation. It canonicalizes exact exponents, groups mathematically equal weights, and simplifies polynomial coefficients under assumptions. This is precisely where aggressive caching and cheap structural equality must be used carefully: replacing exact equality of algebraic weights by floating-point equality would invalidate the support and frontier logic.

The independent oracle checked the logarithmic example

$$f(x) = x + x^2(1 + \log x) \quad (8)$$

through six inverse powers. With $L = \log y$,

$$\begin{aligned} g(y) = & y - (1 + L)y^2 + (2L^2 + 5L + 3)y^3 \\ & - \left(5L^3 + \frac{41}{2}L^2 + 27L + \frac{23}{2} \right) y^4 \\ & + \left(14L^4 + \frac{241}{3}L^3 + \frac{335}{2}L^2 + 151L + \frac{299}{6} \right) y^5 + \dots \end{aligned} \quad (9)$$

Substituting the independently generated polynomial into the forward expression and expanding formally cancels the residual through the checked order. This is a substantive formula check, but it does not exercise Wolfram's pattern dispatch, holding attributes, or assumption machinery.

3.2 Why finite exponent truncation is well defined

If $\delta_* = \min_i \delta_i > 0$, then

$$\mathbf{k} \cdot \boldsymbol{\delta} < H \implies |\mathbf{k}| < H/\delta_*.$$

Only finitely many multi-indices contribute below a finite cutoff. This argument works for irrational gaps as well as rational ones; it does not require a lattice with a common rational denominator.

For a target exponent cutoff h , the retained relative weight condition is

$$\sigma < H, \quad H = |p|h - r_{\text{loc}}, \quad (10)$$

where $r_{\text{loc}} = r$ for a power of the positive finite-endpoint displacement and $r_{\text{loc}} = -r$ when an original-variable power is expressed at source infinity. The distinction between a source-side sign, an observable power, and the local-coordinate exponent should be present in explicit metadata, not reconstructed from a descriptive string.

The article's boundary lemma is sound in the reviewed setting: because every gap is positive, every predecessor of an index at the least omitted weight has a strictly smaller weight. Every contribution to that first omitted weight is therefore present in the one-step boundary of the downward-closed retained index set. The conclusion does not say that every *later* omitted weight can be reconstructed from the same boundary without further work.[20]

A minor editorial correction would improve the depth proof: the number of multi-indices of total degree n is $\binom{n+m-1}{m-1}$, not literally m^n . The latter is a valid upper bound for $m \geq 1$ and is sufficient for the majorant argument. Writing "bounded by m^n " avoids turning a useful estimate into an incorrect equality. This does not invalidate the intended convergence estimate.[20]

3.3 Transport of an unknown forward remainder

Suppose the finite model is perturbed by R with

$$R(u) = \mathcal{O}(u^\rho \mathcal{M}(u)^k), \quad R'(u) = \mathcal{O}(u^{\rho-1} \mathcal{M}(u)^k), \quad \rho > p. \quad (11)$$

Near the chosen branch, $f'(u) \asymp u^{p-1}$ and both inverses are comparable with z . The mean-value theorem gives

$$G(y) - g(y) = \mathcal{O}(z^{\rho-p+1} \mathcal{M}(z)^k), \quad (12)$$

and applying the derivative of a fixed power gives

$$G(y)^r - g(y)^r = \mathcal{O}(z^{\rho-p+r} \mathcal{M}(z)^k). \quad (13)$$

This explains the implemented target precision cap $(\rho - p + r_{\text{loc}})/|p|$. Losing the observable derivative factor would be a real mathematical error; the reviewed constructor explicitly includes it.[21, 3]

The derivative hypothesis in (11) is not a consequence of the value bound. For example,

$$R(u) = u^2 \sin(u^{-2}), \quad R'(u) = 2u \sin(u^{-2}) - 2u^{-1} \cos(u^{-2}).$$

The perturbation is $\mathcal{O}(u^2)$ while its derivative can destroy monotonicity of $u + R(u)$. The repository article makes this point and the guide explains that an explicit `"InputRemainder" -> {rho,k}` declaration includes a matching first-derivative order. This is a positive design decision. The declaration is still a hypothesis supplied by the caller, not a theorem proved about an unspecified function.[21, 2]

The same discipline applies to `SeriesDifferentiate`. Differentiating a finite expression is always an algebraic operation, but differentiating its remainder requires additional information. The repository refuses unsupported derivative claims and does not automatically let a derivative bound declared for an old remainder establish a stronger bound after refinement. That distinction should remain visible in any future convenience interface.[4]

3.4 Newton precision and independent residual checks

The Newton state works with a unit correction U and a verified relative precision. Starting below the least positive gap, the zero correction is a valid seed. Each step raises the working cutoff, typically by doubling, computes the normalized model equation and derivative, applies a truncated Newton correction, and checks that the exact symbolic residual is zero below the newly claimed cutoff.[7]

That residual check is valuable: it verifies the finite-model equation rather than relying on an iteration count or numerical smallness. It also protects reused states. It does not by itself prove that an arbitrary original forward function has the same expansion, nor does it bound unknown forward tails without the separate transport contract. The repository mostly keeps these scopes separate; F05 concerns a misleading normalization label, not a discovered flaw in the normalized model computation.

3.5 Gamma inversion around an exact Lambert core

For increasing Gamma or LogGamma inversion at a source infinity, the package does not try to place every correction directly into powers of $1/y$. Let Y be the transformed logarithmic target and solve the dominant equation

$$H(X) = X(\log X - 1) = Y, \quad X = \frac{Y}{W(Y/e)}.$$

The exact Lambert core is retained. Put $t = 1/X$, $q = 1/\log X$, $C = \log(2\pi)$, and write the Gamma argument as $X(1 + U)$. The finite Stirling equation gives

$$U = \frac{1 - Cq}{2} t + \left(\frac{q}{24} - \frac{C^2 q^3}{8} \right) t^2 + \dots \quad (14)$$

An independent symbolic reversion checked the finite equation through t^4 . Numerical comparisons at several core values are reported in Section 13. The source correctly treats Stirling's expansion as a finite Poincare model rather than a convergent infinite power series.[15, 37]

Retaining a complete polynomial in q at each power of t is mathematically important. For any fixed j and any positive ϵ , q^j decays much more slowly than t^ϵ . Throwing away a higher inverse-logarithmic power at an earlier t -weight cannot be justified by a later t -power cutoff. The coefficient ring and the outer exponent ordering are separate layers.

3.6 Barnes G inversion and a different logarithmic coefficient variable

For Barnes G, with its argument written as $1 + X(1 + U)$, the dominant logarithmic phase is

$$H(X) = \frac{X^2}{2} \left(\log X - \frac{3}{2} \right), \quad X = \sqrt{\frac{4Y}{W(4Y/e^3)}}.$$

Its derivative is $H'(X) = X(\log X - 1)$, so the natural coefficient variable is now $q = 1/(\log X - 1)$, not $1/\log X$. With $A = \log \text{ Glaisher}$ and $C = \log(2\pi)$, the first corrections are

$$U = -\frac{Cq}{2}t + \left(\frac{1}{12} + Aq + \frac{C^2q^2(1-q)}{8}\right)t^2 + \dots \quad (15)$$

The independent finite-model check again extends through t^4 . The shift of the Barnes argument by one is essential and is explicitly represented in the source.[16, 15, 38]

The implementation uses a proved sufficient increasing domain with Barnes argument greater than three. That is a useful safe domain, not a claim to have computed the maximal real monotonicity interval. Generalizing automatic branch selection should not quietly turn a sufficient domain certificate into a maximal-domain assertion. The current provenance wording is appropriately cautious.[16]

4 Comparison with built-in Wolfram Language

4.1 The comparison boundary

Three things must be compared separately: the class of mathematical problems a system can solve, the representation of a returned approximation, and the evidence retained with that approximation. A broader native solver may produce a useful result without the package’s reusable state or explicit error descriptor. A structured package result may carry better provenance while accepting fewer inputs. Neither observation establishes universal superiority.

The official documentation consulted here describes native **Series**, **Asymptotic**, **InverseSeries**, and **AsymptoticSolve**. Their scope is not confined to analytic Taylor expansions. In particular, **SeriesData** supports fractional powers on a rational lattice, and its coefficients can contain logarithms. Limitations of that single data head must not be confused with limitations of all native asymptotic computation.[27, 28, 29, 31]

4.2 Capability matrix

Area	Native Wolfram capability	Asymptotic 1.8.0 and assessment
Taylor, Laurent, Puiseux	Series , SeriesData , InverseSeries , and ComposeSeries provide mature native operations.	Substantial overlap. The package adds explicit generalized remainder and provenance conventions, rather than inventing local series arithmetic.[27, 31, 32, 3]
Irrational power support	Native expressions and asymptotic functions are broader than the rational exponent grid inside one SeriesData .	A finite exact real exponent semigroup is a first-class internal support with complete-block cutoffs and inverse coefficients. This is a representation and algorithmic advantage, not a theorem that native tools always fail.[28, 29, 3]
Logarithmic coefficients	Native series can contain logarithmic coefficients; Asymptotic supports non-Taylor scales.	Polynomial logarithms, their complete weight blocks, and logarithmic remainder degrees are explicitly managed. More elaborate logarithmic hierarchies use distinct package routes.[28, 29, 2]

Area	Native Wolfram capability	Asymptotic 1.8.0 and assessment
Implicit equations and branches	AsymptoticSolve handles equations and systems, real or complex centers, conditions, and inferred scales.	More specialized scalar inverse-germ constructors with selected source branches, retained models, and explicit evidence. The package is not a replacement for the general implicit-system solver.[30, 2]
Exact inverse cores	Closed forms, ProductLog , symbolic manipulation, and series operations can be combined manually.	Dedicated core perturbation paths preserve an exact inverse while expanding corrections and recording the relevant hypothesis and remainder structure.[2, 19]
Gamma/Barnes inverse	Native special functions, their asymptotics, and general solving tools supply useful building blocks.	Dedicated ordered reversion around an exact Lambert core, with a specific inverse-logarithmic coefficient ring and source-domain provenance. No native head-to-head success claim is made without execution.[15, 16]
Forward special functions	Extensive native special-function series are directly available.	The new importer explicitly uses native Series ; its extra work is safe tree interpretation, real-domain validation, carrier separation, and error transport.[27, 13]
Oscillatory coefficients	Native symbolic trigonometric and asymptotic machinery can express oscillations.	A finite exact Fourier-polynomial algebra is supplied for a specified inverse model; frequency limits are resource limits, not silent spectral truncation.[17]
Exponential sectors	Native expressions and asymptotics can retain exponential factors.	Dedicated finite flat-sector inversion preserves an exact zero sector and a separate exponential truncation degree. It is not a general resurgent transseries engine.[18, 2]
Arithmetic on results	Native series have arithmetic, composition, and calculus.	Ordered arithmetic where supported; conservative composite envelopes when no shared ordered algebra applies. Some operations remain scale-specific.[28, 4, 6]
Refinement	Recompute a native approximation to another order.	Stored inverse state and replayable recipes enable reuse and stronger provenance, but F04 exposes a missing achieved-precision check in derived replay.[8, 4]
Numerical evidence	FindRoot and arbitrary-precision arithmetic are available.	A branch-aware comparison wrapper adds reported residuals and remainder-scale ratios; it is explicitly not a certificate.[12]
Exact root certificate	Requires an appropriate verification method; an asymptotic expression alone is not such a method.	A dedicated rational interval evaluator can prove a unique root in a supplied interval for admitted explicit elementary expressions. Its scope is intentionally narrower than the full package.[11]
Integration, differential equations, multiple variables	AsymptoticIntegrate , AsymptoticDSolveValue , and multivariable native facilities address these problems.	No comparable general subsystem is established by this audit. They are possible extensions, not current parity claims.[33, 34, 30]

4.3 Where the package is genuinely differentiated

Its clearest differentiation is the combination of a specialized exact coefficient calculus with explicit precision and branch bookkeeping. For a finite power-log inverse, the package can expose the normalized model, individual multi-index coefficients, complete weights, omitted frontier,

and a retained computation state. Those are useful interfaces for an implementer or researcher inspecting why a coefficient is present and how a remainder was obtained.[3, 7, 8]

The Gamma and Barnes designs are also more specific than asking a general solver for several terms. They declare a two-layer ordering: a core-power exponent outside, and a complete inverse-logarithmic polynomial inside. Likewise, flat and Fourier variants explicitly state which perturbations are admitted and which additional degree or frequency budget applies. These are useful mathematical contracts even when a native combination of functions can recover equivalent formulas for a particular input.[15, 16, 18, 17]

The exact certificate subsystem is a third differentiator, but its value depends on keeping it narrow and truthful. It verifies the stored explicit equation on a supplied rational interval, not a symbolic family of all possible functions hidden behind an unspecified order term. It does not certify every special-function inverse merely because those inverses use the same public result head.[11]

4.4 Where native capabilities should be reused rather than recreated

For ordinary rational-grid series with no special evidence requirement, using the native series algebra as a reference or fast path is sensible. For special functions, the repository already makes the right architectural move: use native structured expansions, retain their order information, and place a checked adapter around them. Additional function coverage should primarily improve this boundary rather than duplicate native special-function tables.[13]

The comparison program supplied in `wolfram/CompareBuiltins.wls` collects outputs for ordinary, logarithmic, and irrational inverse examples, Bessel forward asymptotics, Gamma inversion, and the Newton enumeration example. It records output heads, messages, timeouts, and kernel identity. It deliberately does not assert that all native cases succeed or compare raw numeric order parameters as though every function uses identical semantics.

A fair numerical comparison must first match the branch and independent variable, then convert to a common coordinate and equivalent retained precision. A count of three Bessel amplitude blocks is not necessarily three terms in a fully expanded trigonometric expression. A cutoff of five inside an exponential prefactor is not an absolute $\mathcal{O}(x^{-5})$ error. An unexecuted or unevaluated native command is not evidence that the mathematical task is impossible for Wolfram Language.

5 Concrete findings and proposed corrections

5.1 F01: An optional dense export can dominate the entire computation

Classification: source-confirmed resource defect; high priority.

Location. `makeSeriesData` and `makeInverseSeriesData` in `AsymptoticInverse/Kernel/AsymptoticInverse.wl`; inverse refinement also calls the latter.[3, 8]

Both conversion helpers compute a common denominator for the rational exponents and the remainder boundary, then allocate one coefficient slot for every intervening lattice point. The decisive operations are structurally

```
den = LCM @@ (Denominator /@ Append[exps, remData[[1]]]);
nmin = Min[exps] den;
nmax = remData[[1]] den;
coeffs = Table[0, {nmax - nmin}];
```

The forward helper has an additional empty-support case, but the allocation problem is the same. The `SeriesData` property is constructed eagerly as part of making the public object, not only when a caller explicitly requests a dense export.

Consider the mathematical input

```
AsymptoticExpansion[
  x^(1/1000000000) + x,
  {x, 0, 1},
  "MaxTerms" -> 10
]
```

There is one retained block, $x^{1/10^9}$, and one first omitted block, x . The generalized result is tiny. The dense lattice denominator is 10^9 , however, so the export requires

$$n_{\max} - n_{\min} = 10^9 - 1 = 999,999,999$$

slots. The independent oracle evaluates this integer only; it does not allocate the list. The public reproduction with that denominator should not be run without a protective memory bound. The supplied native regression uses the much smaller denominator 100003 while testing the same policy failure.

Why this matters. "MaxTerms" cannot be understood as a useful resource control when a result with one sparse term triggers a billion-element convenience representation after the mathematical work has finished. This affects both reliability and composability: a caller might never intend to use `SeriesData` at all.

Tactical correction. Check the prospective length before `Table`; if it exceeds a separate dense-export limit, store a structured `Missing["DenseRepresentationTooLarge", ...]` value while preserving the valid sparse result. The supplied patch generator inserts a fixed 20,000-slot safeguard at both allocation sites. This is intentionally a narrow stopgap, not a complete public resource-policy implementation.

Preferred design. Make dense conversion lazy and explicit, with an option such as "MaxDenseSeriesLength". Record its prospective length and rational lattice without allocating. A sparse `SeriesData`-like internal representation should not masquerade as the native head; keep the generalized object and offer a checked conversion operation.

Acceptance criteria. The large-denominator input returns its sparse result without a large transient allocation; asking for the dense conversion either succeeds within budget or returns a diagnostic identifying the required length. The same rules apply to construction, refinement, serialization, and formatting. A missing optional representation must not turn a valid generalized expansion into a failed mathematical computation.

5.2 F02: A nested fractional observable can bypass the real-branch guard

Classification: source-traced correctness defect; high priority; native reproduction pending.

Location. The shared `fwdPower` primitive in the kernel entry file, together with `seriesPower` and `seriesJetApply` in `SeriesOperations.wl`. [3, 4]

The direct power interface checks that a noninteger power has an adequate leading term and a proved real branch. In contrast, the generic observable evaluator can recursively call the lower-level forward power primitive. When that primitive receives a pure remainder, it has a special positive-power branch that transforms the order without establishing the sign of the unknown quantity.

The minimal proposed reproduction is

```
s = AsymptoticExpansion[x, {x, 0, 1}];
(* Finite part: 0. Remainder: O(x). *)

SeriesPower[s, 1/2]
SeriesObservable[s, 1 + Sqrt[-z], z]
```

The first operation is expected to be rejected because the available jet has no nonzero leading block. The second contains the same branch obligation inside a larger expression, but its recursive path reaches `fwdPower` directly. The reviewed pure-remainder branch can produce a retained constant plus a square-root order without rejecting the missing real branch.

The original represented source in this example is $x > 0$. Its observable $1 + \sqrt{-x}$ is $1 + i\sqrt{x}$ on the principal complex square root, not a real observable on the selected source approach. Even apart from that explicit source, the input jet $\mathcal{O}(x)$ alone proves neither eventual positivity nor eventual negativity. An order estimate cannot supply the missing branch proof.

This finding is not the statement that $|\sqrt{-x}| = \mathcal{O}(x^{1/2})$ is false; that magnitude estimate is true. The defect is returning through a *real-branch* computation interface without satisfying its branch contract. Such a result can later be mistaken for an admissible real input by another operation.

Correction. Move the required pure-remainder protection into `fwdPower`, the shared semantic primitive. For an empty finite support with finite remainder precision, reject noninteger powers unless a separate domain witness is supplied. The narrow patch is

```
If[P != Infinity && ! IntegerQ[rr],
  fail["UnknownLeadingTerm",
    "A pure remainder does not establish the real branch required by a noninteger power."],
];
```

inside the existing `T === {}` branch. Exact zero with a positive exponent remains a separate admissible case. Nonpositive powers of a pure remainder remain rejected. An adaptive forward constructor can respond to the missing-leading-term failure by obtaining more input order, rather than inventing a branch.

Acceptance criteria. Direct powers, nested observables, ordinary arithmetic normalization, and composition all enforce the same rule. Positive leading terms still allow square roots. Unknown sign remains unknown. An explicitly complex future mode must use a different domain contract rather than quietly relaxing the real one.

5.3 F03: The constructor defeats part of the Newton and grouped-Lagrange advantage

Classification: source-confirmed algorithmic bottleneck with an exact independent count; high priority.

Location. The main kernel functions `construct`, `indexRegion`, and `inverseFrontier`; the Newton and grouped-Lagrange routines in `IncrementalInverse.wl`.^[3, 7]

The Newton and grouped routines are not merely names for the same inner algorithm. Newton carries a verified precision and doubles it; grouped Lagrange combines support at a common depth and weight before applying the Euler operators. However, ordinary construction still builds a multi-index region before dispatching to these routines. Frontier construction also relies on index-based coefficient work. A term-goal request first advances an incremental Lagrange state and can then recompute with the requested alternative method.

Take

```
f = Sum[x^k, {k, 1, 31}];
AsymptoticInverse[f, {x, 0}, {y, 31},
  Method -> "Newton", "MaxTerms" -> 20000]
```

Here $p = r = 1$, the gaps are $1, 2, \dots, 30$, and the retained relative weight bound is $H = 30$. The number of nonnegative integer vectors with

$$k_1 + 2k_2 + \dots + 30k_{30} < 30$$

is exactly 23,025. This is already over the stated 20,000 index budget before boundary entries are added. In contrast, the requested inverse has only 30 ordinary power positions. The support count is independently verified by an integer generating-function dynamic program.

There is also a simple reference inverse to the requested order. Since

$$f(x) = \frac{x}{1-x} - \frac{x^{32}}{1-x},$$

its inverse satisfies

$$g(y) = \frac{y}{1+y} + \mathcal{O}(y^{32}).$$

Thus all requested coefficients through y^{30} are the alternating coefficients of $y/(1+y)$. The example is not intrinsically a difficult inverse problem; its difficulty is induced by an avoidable combinatorial representation.

Correction. Separate three services: computation of the retained inverse jet, discovery of reachable weight layers, and computation of an optional complete first omitted coefficient. Newton should need a truncated equation algebra and a support/precision policy, not the entire Lagrange multi-index region. Grouped Lagrange should operate on aggregated powers of the perturbation throughout construction, including its frontier strategy.

A conservative remainder based on a verified residual and the known model need not have the exact first omitted coefficient eagerly attached. Return a justified coarse bound when an exact frontier is expensive, and compute the sharper frontier lazily or on request. This trades optional sharpness for predictable work without changing retained coefficients. Any such change must still honor the transported forward remainder cap.

Acceptance criteria. The example above does not allocate or enumerate all 23,025 multi-indices on the Newton path. Statistics distinguish equation operations, support layers, and any optional frontier coefficient work. Increasing `SeriesTermGoal` with Newton should not force a hidden full Lagrange calculation first. Agreement tests compare complete coefficients, not only numerical values at one point.

5.4 F04: Derived refinement can return a much weaker expansion than requested

Classification: source-traced API/precision-contract defect; high priority; native reproduction pending.

Location. Derived recipe replay in `SeriesRefine` and the wrapper `seriesRefinementResult` in `SeriesOperations.w1.[4]`

The replay path refines each operand with a fixed guard margin based on the requested cutoff and a coarse property of the old operand. It then reruns the requested operation. The operation

correctly clips its precision to what the operands justify, but the surrounding refinement wrapper does not require that the resulting precision reaches the requested cutoff.

A concrete example is

```
s = AsymptoticExpansion[Exp[x] - 1, {x, 0, 3}];
t = SeriesPower[s, -10];
r = SeriesRefine[t, 5];
{r["Cutoff"], r["RemainderPower"]}
```

The input has valuation $\alpha = 1$. For a nonzero fixed power r_0 , a perturbation of order P changes the observable by order

$$P + (r_0 - 1)\alpha. \quad (16)$$

With $r_0 = -10$, this is $P - 11$. The current fixed guard requests operand order $5 + 2 = 7$ in this case. That justifies a result only through a remainder of order $7 - 11 = -4$, not order 5.

The missing accuracy is not marginal. The source $e^x - 1$ truncated before x^7 differs by $x^7/7! + \dots$. The derivative of its negative tenth power has leading factor $-10x^{-11}$, so the leading propagated discrepancy has magnitude proportional to $x^{-4}/504$. The exact source is available for replay; this is not an unavoidable input-precision cap.

Important distinction. The returned weak remainder can be mathematically valid. The defect is that a call named “refine to five” succeeds with a cutoff label of five while achieving far less. This is different from fabricating higher-order coefficients or claiming the error is $\mathcal{O}(x^5)$. It is still serious because adaptive clients naturally use successful refinement as a postcondition.

Correction. Implement backward precision requirements. For a power observable with a proved nonzero branch,

$$P_{\text{input}} \geq h - (r_0 - 1)\alpha. \quad (17)$$

The example needs input order at least 16. Analogous rules apply to logarithm, exponential, multiplication, differentiation, and composition, with coefficient-logarithmic and carrier information where necessary.

A less invasive first correction is an adaptive replay loop: after each attempt, compare the *achieved* precision with the request; obtain more operand precision until the target is reached or a source/resource limit prevents it. Failure should return the best valid expansion together with requested and achieved precision. The supplied surgical patch intentionally does not attempt to redesign this planner.

Acceptance criteria. A successful scalar-cutoff refinement proves its postcondition in the recorded scale. The negative-power example reaches at least order five. Resource failure or an explicit source cap produces a structured failure with “BestExpansion”; it does not silently return an under-refined success. The same check applies to composite recipes with more than one operand.

5.5 F05: Residual normalization text loses the target offset

Classification: source-confirmed diagnostic/documentation defect; low priority.

Location. `InverseResidual::usage` and the ordinary residual report’s “Normalization” string in the main kernel.[3]

The reported formula is written as $f(g(y))/(az^p) - 1$. For a shifted target limit, the actual normalized equation uses

$$\frac{f(g(y)) - y_0}{az^p} - 1. \quad (18)$$

The internal model computation and the repository's transport article account for the offset; the descriptive string does not. For $f(x) = 7 + x$ and $g(y) = y - 7$, the computed residual is zero, while the unshifted displayed formula would be $7/(y - 7)$.^[21]

The proposed patch changes both textual occurrences to include y_0 . A later API should expose normalization as structured symbolic data, including source function, target offset, denominator, and scope, rather than requiring a caller to parse a prose formula.

5.6 F06: Exact logarithm canonicalization can trigger unbounded factorization

Classification: source-confirmed performance risk; medium priority; no native timing claim.

Location. `logCanon` in the main kernel.^[3]

The canonicalizer decomposes logarithms of positive integers or rationals using `FactorInteger`. This can produce pleasant prime-logarithm normal forms, but exact factorization is a qualitatively different resource task from polynomial coefficient manipulation. A compact integer argument with difficult factors can make simplification dominate a small expansion. The ordinary term budget does not bound the factorization work.

The recommendation is not to abandon useful identities such as $\log 12 = 2 \log 2 + \log 3$. Use cheap perfect-power and small-factor normalization, a bounded factoring policy, or the documented easy-factor mode, and retain an opaque `Log[n]` when further work is not justified. Exactness does not require a fully factored integer inside every logarithm. Wolfram documents partial/easy factorization controls; the choice should be explicit rather than an accidental consequence of formatting coefficients.^[36]

A local cache may avoid repeated work, but caching a hard call does not protect the first call. Bound expression size, integer bit length, and elapsed symbolic work separately. Do not advertise a strict runtime bound merely because an option limits a number of factors; the public budget must cover the actual computation strategy.

Acceptance criteria. An expansion containing a large exact logarithmic constant can return an exact result with that constant still opaque. Expensive normalization is optional, interruption-safe, and reported in statistics. Two algebraically equivalent coefficient forms do not require equal display forms to be accepted as equal by every operation.

5.7 F07: The flat-rate test is narrower than commensurability

Classification: documented limitation and misleading failure classification; medium priority.

Location. `flatModel` in `FlatSectors.wl`.^[18]

The code chooses the smallest supplied phase rate as its base and requires every other rate divided by that base to be a positive integer. Consequently,

```
AsymptoticFlatInverse[
  x + Exp[-2/x] + Exp[-3/x], {x, 0}, {y, 3}]
```

is rejected by the rate test, even though 2 and 3 are commensurate. They are both integer multiples of the primitive base 1, which need not itself occur as an input rate. The existing failure message states the implemented restriction, but the tag `"IncommensurateFlatRates"` describes a stronger mathematical conclusion than was established.

For positive rates c_i whose ratios to a chosen c_0 are rational, compute

$$q_i = \frac{c_i}{c_0}, \quad D = \text{lcm}_i \text{den}(q_i), \quad n_i = Dq_i, \quad d = \gcd_i n_i. \quad (19)$$

Then

$$c_* = c_0 \frac{d}{D}, \quad k_i = \frac{n_i}{d} \quad (20)$$

gives a positive primitive base and positive integer sector degrees. For rates $2/3, 5/6$, the base is $1/6$ and the degrees are 4, 5. The independent code checks both examples; `ProposedHelpers.wl` supplies a native implementation proposal without modifying the package.

The resulting degrees must be budgeted before expansion. A tiny primitive base can produce a huge largest degree even for a short rate list. Irrationally independent rates are a separate future problem requiring multiple markers or an ordered rate semigroup; they should not be forced into this rational-lattice extension.

Acceptance criteria. Rationally commensurate positive rates use a primitive base with correct degree metadata and unchanged physical exponentials. Unsupported irrational ratios and excessive degree requirements produce different failures. Existing inputs whose smallest rate already generates the lattice preserve their results.

6 Performance and resource engineering

6.1 Separate the relevant measures of work

A single option named `"MaxTerms"` currently stands in for several different resources: input expression leaves, retained convolution pairs, multi-index region size, polynomial expression growth, iteration counts, or sector complexity. These quantities can differ by orders of magnitude. A bound that is adequate for one is not automatically meaningful for another. The dense export and factorization findings are examples of work that escapes the intended small-problem intuition entirely.[3, 7, 17, 18, 13, 6]

A better resource record would carry separate budgets and counters:

```
"ResourceBudget" -> <|
  "SupportEntries" -> 20000,
  "CandidatePairs" -> 200000,
  "ExpressionLeaves" -> 200000,
  "DenseExportSlots" -> 20000,
  "CoefficientBitLength" -> 1000000,
  "SymbolicTime" -> 60,
  "ProofTime" -> 15
|>
```

These values are illustrative design parameters, not proposed new defaults. The important point is the separation. A failed branch proof should not be reported as support exhaustion. Exceeding a display-conversion budget should not discard a valid sparse expansion. Exhausting an optional simplification budget should preferably leave an exact but less simplified coefficient.

Resource accounting should also be compositional. Eight attempts with a 30-second native series limit, interleaved with several bounded simplifications, do not amount to one 30-second public-call limit. A shared deadline or remaining-work context is more predictable than independently resetting a timeout in every helper. Child operations should report how much budget they consumed and why they stopped.[13, 6]

6.2 Avoid expression growth before checking the budget

Several routines correctly check prospective pair counts or support size before producing large intermediate results. The Fourier source-weight convolution, for example, uses sorted weights to skip columns that are already beyond the exclusive cutoff. The composite error normalizer estimates distributed-error expansion size before expanding its remainder factors. These are good patterns to extend.[17, 6]

By contrast, a check of `LeafCount[result]` after `Expand` protects only the final return path; it does not prevent a very large intermediate expansion. `flatTrim` begins by expanding its expression before enforcing its resulting leaf-count budget. Similar risks arise from repeated coefficient-polynomial powers and native integer powers of uncertain expressions.[18, 13, 7]

Prefer truncated polynomial multiplication and degree-aware extraction to fully expanding and then truncating. In flat-sector arithmetic, the marker polynomial can be represented by degree-to-coefficient rules so that products with sector degree above the requested depth are never built. Coefficients still need their own expression budgets, since a small number of marker degrees can hide enormous expressions.

For nonnegative integer powers of a native uncertain expression, repeated multiplication by the same argument is linear in the exponent. Binary powering reduces the multiplication depth, provided each intermediate error envelope is transported correctly. This change should be checked for expression swell and not justified solely by the number of multiplications: a different multiplication order can produce very different symbolic simplification costs.[13]

6.3 Weight queues and collision-aware support

The incremental inverse state stores a boundary association, selects the complete next weight layer, generates neighbors, and finds the next minimum by sorting boundary weights. It also builds an explicit list of processed indices and appends new blocks. This is clear and correct in the reviewed positive-gap setting, but repeated full scans and sorting become expensive when the frontier is wide.[7]

An exact-weight priority queue with a bucket of all equal weights can remove repeated global minimum searches. It must preserve complete collision layers: popping one minimal item and immediately emitting a coefficient is not enough when another multi-index has the same exact weight. For algebraic weights, canonicalization and comparison may cost more than the queue operations themselves, so performance counters should distinguish those costs.

A more consequential optimization for commensurate gaps is to choose a primitive rational weight lattice and use an aggregated coefficient polynomial indexed by a single integer weight. This can turn the many-generator model in F03 into ordinary polynomial operations. For incommensurate gaps, sparse exact-weight support is still appropriate; a lattice fast path should not weaken the general implementation's exact ordering guarantees.

The grouped Lagrange representation $(n, \sigma) \mapsto A_{n, \sigma}(L)$ is another useful middle ground. It aggregates all multi-indices with the same depth and weight before applying common Euler operators. A strategy selector should estimate expected collision density, number of generators, desired cutoff, and coefficient degree before choosing among direct indices, grouped coefficients, and Newton. Choosing a method based only on the desired number of displayed blocks misses the dominant cost drivers.

6.4 Retained state is useful, but its cost should be visible

The refinement subsystem already has several strong properties: state is returned by value, model/branch/assumption signatures are checked, displayed coefficient prefixes are compared against stored state, and Newton states are verified by residual computation. A failed refinement does not mutate the caller’s prior object. This is much safer than an invisible process-global cache keyed only by a printed expression.[8, 7]

Nevertheless, retaining every processed index, coefficient block, polynomial power, history entry, and nested recipe can make object size grow much faster than its visible approximation. The current polynomial-power cache has an explicit capacity derived from the index budget, which is helpful; coefficient expression size and recipe graph size need comparable visibility.[7, 8]

Recommended operations are a compact result view, an explicit `ForgetComputationState` or equivalent option, a state-size report, and a versioned serialization format. Recipe storage should favor a shared directed acyclic graph rather than repeated nested value copies when operands recur. These are design proposals, not measured memory leaks in the current implementation.

6.5 Interpreting the repository’s existing benchmarks

The validation documentation reports several targeted optimization measurements, with exact-output equality checks and scoped methodology. Those are useful evidence for the changes they measure. They are not fresh benchmark results from this audit.[22]

For example, the recorded Fourier convolution microbenchmark improves from about 1.28634 seconds to 0.00776 seconds, whereas the reported public Fourier inverse fixture changes from about 0.02514 to 0.02325 seconds. A large speedup in one helper therefore does not imply a comparable end-to-end improvement. Similarly, a coordinate helper’s substantial speedup says little about an input dominated by symbolic normalization or native special-function expansion.[22]

Future benchmark reports should contain the precise source hashes, kernel/build and operating system, input family and parameter size, cold versus warm protocol, output equivalence check, and separated timing categories. A meaningful scaling series is more informative than one impressive ratio. For F03, report generators, output weights, processed indices, frontier size, and total time as the cutoff grows. For F01, report the prospective dense length while confirming that no allocation took place.

7 Specialized scales: strengths, gaps, and extension boundaries

7.1 Native special-function import is an adapter, not an independent oracle

The native importer calls `Series` in a positive local coordinate with `Analytic -> False`, inspects the returned expression tree, and propagates absolute error information through supported operations. It has explicit rules for sums, products, powers, exponentials, and trigonometric or hyperbolic functions. Unresolved native expansions and unsupported trees are rejected rather than silently normalized to their finite part.[13]

One particularly careful choice concerns logarithmic tails in native `SeriesData`. The native head gives a rational exponent boundary but does not separately specify the degree of a possible unknown logarithmic polynomial. The importer initially weakens that boundary by half a lattice step. Any fixed logarithmic polynomial is absorbed by such a positive power margin. It then computes a candidate omitted block and verifies that the imported error is negligible compared with the chosen returned bound.[13]

This is conservative and mathematically intelligible. It is not an exact reconstruction of every possible native order convention, nor does it cover arbitrary iterated logarithmic or oscillatory tails merely because they fit syntactically inside a coefficient. An accepted input should retain the normalization rules and native order used, so that later refinement can replay the same interpretation.

The real projection is also important. The source’s real domain is established before complex-looking native expansions are combined into a real answer. Oscillatory amplitudes retain bounds rather than being treated as nonzero leading coefficients eligible for division. This avoids a common error: a sine or cosine coefficient is bounded, but it is not bounded away from zero.

The development opportunity is a diagnostic adapter interface. When import fails, report whether the obstruction was an unproved real domain, an unsupported coefficient algebra, a nonvanishing phase error, a native timeout, or an unresolved native head. Currently these distinctions exist in several failure tags, but a stable capability/dispatch trace would make them easier to discover. The dynamically recognized native numeric-function heads should also be treated as candidates, not as a promise that every such function is supported.[13]

7.2 Gamma and Barnes operations need a reusable coefficient-ring abstraction

The ordinary series-data extractor rejects Gamma/Barnes inverse scales because they are not polynomial in the ordinary logarithm variable. Specialized operations handle some tasks, and the composite-envelope path can retain others without asserting one common ordered coefficient algebra. This is a reasonable conservative fallback, but it makes the behavior of a common public operation strongly dependent on the result’s hidden family.[4, 2, 6]

Rather than adding another collection of family-specific overloads for each new function, define a coefficient-algebra interface: exact addition and multiplication, derivation with respect to the core variable, equality/canonicalization, valuation or dominance information, and a rule for transporting the coefficient remainder. For Gamma, coefficients are polynomials in $q = 1/\log X$; for Barnes they are polynomials in $q = 1/(\log X - 1)$. Their derivative rules are simple but different from treating q as an independent constant.

For example,

$$\frac{d}{dX} \frac{1}{\log X} = -\frac{q^2}{X}, \quad \frac{d}{dX} \frac{1}{\log X - 1} = -\frac{q^2}{X}.$$

The formulas look the same in (X, q) coordinates, while the exact relation defining q differs. Encoding that relation and a derivation on the coefficient ring would support checked calculus and composition without flattening prematurely to large expressions in the target variable.

A native `SeriesData` export should remain optional for these families. Forcing the inverse-logarithmic polynomial into a finite rational power grid would discard the very ordering the dedicated constructor preserves.

7.3 Fourier coefficients: exact closure within a deliberately finite class

A Fourier coefficient has the form

$$\sum_{\omega \in \Omega} P_{\omega}(L) e^{i\omega L},$$

with finite exact real frequency support. The Euler derivation acts on each mode by $P_{\omega} \mapsto P'_{\omega} + i\omega P_{\omega}$. Frequencies add under multiplication, and the implementation checks real-conjugacy relations rather than using a numerical real-part heuristic.[17]

The explicit `"MaxFrequencies"` budget is a strength. It is a hard failure threshold, not a hidden approximation that drops high-frequency modes. This matters for exact cancellation and remainder bounds. A future approximate Fourier mode should be a distinct operation with an error contribution for every discarded mode, not a reinterpretation of the existing budget option.

The present inverse class requires a monomial leading core and strictly higher source-power perturbations. That avoids inversion of a leading oscillatory coefficient that might vanish infinitely often. Extending the class to nonvanishing periodic leading factors would require a proof of a uniform lower bound and potentially a different inversion parameterization. Merely allowing more function heads in the parser would not establish that theorem.

Near-term improvements are automatic dispatch when a supported finite Fourier structure is recognized, complete-block term goals for that family, better reporting of frequency growth, and grouped convolution when many sums of frequencies coincide. More general quasi-periodic or infinite Fourier coefficients should follow only after a summability and tail contract is designed.

7.4 Flat sectors: preserve the exact zero sector

The flat inverse engine admits finite terms built from exponentials such as e^{-c/u^p} around an exact shifted monomial core. The sector degree is independent of the algebraic/logarithmic structure of the coefficient. The omitted-sector remainder carries an asymptotic majorant with existential constants and a sufficiently small neighborhood, not a computable numerical certificate.[18]

The restriction to an exact zero sector is mathematically sensible. A finite algebraic approximation with an untracked algebraic error cannot stand in for an exact zero sector when exponentially small terms are being resolved: every fixed positive power error dominates e^{-c/u^p} near zero. Treating the approximate core as exact would assign meaning to exponentially small corrections that are already overwhelmed by a larger unknown error.

After the primitive-rate improvement in F07, a useful extension would be flat perturbations around a *proved exact nonmonomial inverse core*. This could reuse the exact-core perturbation machinery, but it requires an explicit statement about analyticity, derivative growth, and how core evaluation affects the sector coefficient algebra. Incommensurate rates can then be represented by a multi-marker support, with a well-defined order and a frontier majorant. Neither extension should be described as general transseries support until those contracts are implemented.

7.5 Logarithmic coordinates and special-function adapters

The public guide describes logarithmic hierarchies, reciprocal-logarithmic operations, source/-target coordinate transformations, and special-function inverse adapters. These are substantial parts of the current surface. Their presence makes a unified chart description more valuable: a caller should not need to infer the independent small variable from which constructor happened to produce the object.[2]

For every transformed result, retain the original source expression, the coordinate substitution, the transformed equation, the selected real domain, the reconstruction map, and the exact relation between the requested and returned cutoff. Composition and numerical checks should consume that information through a common interface. Reconstructing a branch from only a printed prefactor or a guessed sign is fragile when several coordinate layers are involved.

The reviewed public dispatch includes applied `InverseFunction` expressions, explicit branch selections, and provenance caches scoped to the call. It tries to establish the source branch rather than treating native series of an unevaluated inverse operator as a coefficient oracle. This is

a good foundation. The remaining challenge is to make branch proof cost and failure reasons visible and to keep equivalent callable and expression forms on the same semantic path.[14]

8 Certificates and numerical verification

8.1 What the exact certificate actually proves

The certificate routine requires an ordered closed interval with exact rational endpoints and an exact numerical target. It checks the selected source side and every retained source-domain condition over the whole verification interval. Its arithmetic uses rational enclosures, directed dyadic rounding, and explicit elementary-function series tails. Floating-point calculations may select a seed, but the final containment and sign predicates are exact.[11]

Let the derivative enclosure avoid zero and let $\mu > 0$ be a lower bound for its absolute value. If the residual at a rational center c is enclosed with absolute bound ϵ , then the candidate radius is

$$R = \epsilon/\mu.$$

Containment of $[c - R, c + R]$ in the verification interval establishes a root through the derivative bound and continuity. Alternatively, exact endpoint signs can establish existence. Strict derivative sign gives uniqueness in the interval. The implementation then sharpens the enclosure using a signed derivative enclosure and the residual correction.[11, 21]

This is not merely checking that a numerical residual is small. The interval must be on the retained branch, the derivative must be separated from zero, and existence needs a valid containment or sign argument. The reviewed implementation addresses these requirements explicitly.

8.2 What it does not prove

The certificate covers the stored explicit forward function. It expressly does not certify every member of a family represented by an unspecified "**InputRemainder**". It is local to the verification interval, not a global inverse-branch theorem. Its admitted elementary expression tree is narrower than the collection of functions that can be expanded asymptotically. In particular, the shared public result head does not make Gamma/Barnes inverses automatically eligible for this certificate route.[11]

The default "**Interval**" $\rightarrow$ **Automatic** can be confusing because the implementation still requires an explicit rational interval. This is an API polish issue: either compute an interval under a documented search policy or use a required argument/clear early diagnostic that makes the obligation obvious. An automatic seed and an automatic verification interval are different tasks.

The result distinguishes absolute and relative requests, uses a certified root-magnitude lower bound when converting a relative tolerance to a sufficient absolute tolerance, and can detect a fixed center's accuracy floor. When absolute and relative tolerances are combined through a maximum, the documentation should state that convention directly; it should not imply that both independent inequalities must simultaneously hold if that is not the implemented stopping rule.[11]

8.3 Numerical checks are useful but should not borrow certificate vocabulary

The numerical checker solves the original stored equation from an expansion-derived seed, checks the selected source side and retained source predicate at the recovered root, and reports the

approximation error relative to a remainder scale. Exact target substitution occurs before numerical evaluation, helping preserve a small target displacement near a large offset. These are thoughtful implementation details.[12]

The `"ExactInverse"` field is a legacy alias for a numerical reference root. The source explicitly says so, so it is not an undocumented claim of exact arithmetic. Nevertheless, the name is easy to misuse in downstream code. Prefer `"ReferenceRoot"`; deprecate the old field with a clear migration note rather than continuing to extend its use.

A small residual alone does not guarantee a small root error when the derivative is small. The diagnostic should retain the actual equation, branch, working precision, residual, and, where available, a numerical conditioning estimate. The exact certificate remains the appropriate route when a rigorous error bound is required and the expression lies in its supported class.

8.4 A credible route to special-function certification

Extending certificates to Gamma and Barnes is worthwhile, but it is a different project from extending formal coefficients. The first stage should enclose the *logarithmic* function and its derivative on a proved positive interval, using known finite asymptotic bounds plus recurrence/argument shifting where helpful. For positive real Gamma arguments, the classical Stirling remainder controls documented by DLMF provide a useful starting point. Barnes requires its own justified remainder and derivative enclosure; it should not inherit Gamma's bound by analogy.[37, 38]

The implementation would need exact enclosures for constants such as π and $\log(2\pi)$, not just arbitrary-precision approximations with unproved rounding. It would also need a careful decision about whether to certify the original function, its logarithm, or a monotone transformed equation. The certificate result should identify that route and the equivalence conditions.

A separate independent certificate checker would strengthen the trust boundary. The generator could return rational interval endpoints, elementary bound witnesses, derivative separation, and endpoint or residual containment evidence. A small checker should validate those witnesses without rerunning heuristic branch discovery or trusting cached metadata. This is a focused and realistic formal-verification target, unlike attempting to verify the whole symbolic system at once.

9 API and user experience

9.1 Make a precision request a typed object

The guide already contains a detailed cutoff table. The problem is not a total absence of documentation; it is that a numeric positional order has too many meanings for programs to safely treat it uniformly. Ordinary absolute exponents, powers inside an exact prefactor, inverse-logarithmic exponents, inclusive marker depth, inclusive flat-sector degree, and per-carrier block goals are all legitimate but different notions.[2]

A proposed machine-readable request might be

```
"PrecisionGoal" -> <|
  "Coordinate" -> w,
  "Measure" -> "PowerExponent",
  "Bound" -> 5,
  "Boundary" -> "Exclusive",
  "Normalization" -> "Absolute"
|>
```

A sector request would instead name the marker or phase degree, its inclusive boundary, and any inner amplitude cutoff. Existing compact syntax can remain as a convenience that resolves to this object. Every result should expose both `"RequestedPrecision"` and `"AchievedPrecision"`, along with a predicate or certificate that the latter meets the former. F04 becomes much harder to hide when success has this explicit postcondition.

A boolean `"GoalReached"` alone is not enough unless its meaning is tied to a scale. A composite error envelope may have several incomparable coordinates and no single power exponent. Such a result should state that the scalar precision query is unsupported rather than fabricate an order from its smallest printed exponent.

9.2 Separate absolute observables from displacement powers

The documented `"Power"` option has an unusual discontinuity. At a finite source endpoint, a nonunit power represents $(x - x_0)^r$, while the default $r = 1$ returns the original source variable including its translation. At infinity it represents x^r . This convention is explained in the guide, but a user can reasonably expect powers to compose in the ordinary way.[2, 3]

For the identity function near $x_0 = 1$,

```
AsymptoticInverse[x, {x, 1}, {y, 3}, "Power" -> 2]
SeriesPower[AsymptoticInverse[x, {x, 1}, {y, 3}], 2]
```

represent $(y - 1)^2$ and y^2 , respectively. That is intentional current behavior, not a discovered wrong coefficient. It should nevertheless motivate a more explicit interface:

```
"Observable" -> x^2
"Observable" -> (x - x0)^2
```

A dedicated `"DisplacementPower"` shorthand can preserve the convenient coefficient formula. A migration should retain the legacy option with a precise warning or compatibility policy rather than silently changing stored results.

9.3 Assumptions should follow one predictable rule

The package's documented default is `Assumptions -> True`. Native assumption-aware functions commonly obtain their default from `$Assumptions`, which is what `Assuming` changes. Consequently, wrapping a package call in `Assuming[a > 0, ...]` is not equivalent to explicitly passing the parameter assumption through the package's public option-resolution path.[2, 3, 14, 35]

This should be treated as an integration mismatch, not as a claim that every internal native simplification ignores ambient assumptions. Some helpers call native functions with explicit assumptions and some normalization routines have their own behavior; that makes inconsistent ambient influence an additional risk. The public contract should resolve assumptions once, freeze them in the result, and pass that resolved context through every operation.

The natural proposed default is `Assumptions :> $Assumptions`, with the same explicit-option override convention users expect from native functions. Source/target conditions still need to be separated from parameter-only assumptions as the current wrappers do. A later `SeriesRefine` should replay under the stored resolved assumptions, not whatever unrelated `$Assumptions` happens to be active then.

Regression tests should compare explicit and ambient parameter assumptions, verify that changes to the ambient context after construction do not change stored-state meaning, and check

contradictory source conditions. Assumption contexts belong in cache signatures and branch evidence.

9.4 Preserve unknown, unsupported, and disproved as different outcomes

A real-branch predicate may be true, false, or not established within the proof budget. An exponent comparison may be decidable, unsupported in the current ordering engine, or timed out. Those outcomes should not collapse to one **False** value in public diagnostics.

The current source contains many useful **Failure** tags, and some branches deliberately return **\$Failed** to mean “try another constructor.” That internal sentinel is reasonable only when callers know which failures are terminal. A resource failure during a promising branch should not be silently reinterpreted as mathematical nonmembership in that branch’s class.[3, 14, 13, 6]

A proposed public diagnostic record should include attempted route, admitted-class test, unproved condition, exhausted budget, and possible fallback. This would make an error actionable: “leading coefficient sign unproved under these assumptions” suggests a different remedy from “coefficient algebra cannot represent this iterated logarithm.”

9.5 Expose capabilities and a schema version

The result head changed from **PowerLogSeries** to **GeneralizedSeries** in 1.8.0, and the guide explicitly tells callers to update patterns and saved input. That is honest migration documentation. It also exposes the need for a stable machine-readable schema, because renaming the public head is not the only way stored associations can become incompatible.[2]

A future result should record “**SchemaVersion**”, “**RepresentationKind**”, a normalized chart, and supported operations. A **SeriesCapabilities[s]** query could explain whether ordered composition, differentiation, refinement reuse, dense export, or certification is supported and what prerequisites remain. This is preferable to discovering capabilities by repeatedly catching unrelated failures.

Public construction of **GeneralizedSeries[assoc]** should either validate invariants or be clearly designated as a low-level unsafe constructor. A malformed association must not accidentally be accepted as a verified state just because it has a familiar head. Migration of saved expressions should validate required fields and avoid granting derivative or branch evidence that was absent in the old schema.

9.6 Display, evaluation, and provenance

The current formatting emphasizes the finite expression and remainder, and **Normal[s]** deliberately drops the remainder. Numeric application **s[value]** evaluates the finite expression; it should not be mistaken for certified evaluation. These behaviors are documented, but a visually simple expression can obscure a rich and sometimes conditional object.[2, 3]

A compact display could show the coordinate, source side, evidence level, and whether a requested goal was reached, with the full model hidden behind an inspection command. Such a display should never trigger expensive canonicalization or dense conversion. Copy/paste and **InputForm** must retain the actual object rather than only the finite approximation.

An optional **EvaluateExpansion** report could return the approximation, a checked domain predicate, the asymptotic remainder scale, and any available numerical or interval evidence. It should not automatically convert an asymptotic scale into an absolute error bound with constant

one. The unknown constant and validity threshold are essential parts of what an order estimate does *not* provide.

10 Testing, packaging, and release discipline

10.1 The existing validation is substantial but scoped

The repository is not untested. Its current validation notes report 305 passing and zero failing focused native tests across 18 selected files for the result-head/adjacent refactoring work, as well as separate standalone and Python-builder checks. They explicitly say that the full package suite was not run for that focused validation. The native environment reported there is Wolfram 15.0.1 on Windows. These are repository-reported results, not this audit's executions.[22, 24]

The shared focused runner checks that files exist, rejects an empty test report, records source hashes, verifies that sources remain unchanged, reports failed outputs/messages, and returns a failing process status on failure. These are useful foundations for reliable release evidence.[23]

The risk is a coverage boundary, especially after a public result-head rename touching many families. Passing 18 selected suites is good evidence about those suites; it does not establish that every old pattern match, serialization path, or rare adapter was updated. A full release gate and targeted pull-request gates solve different problems and both are needed.

10.2 CI should execute semantics as well as bundle freshness

The inspected GitHub workflow checks standalone-source freshness and Python tests for the standalone builder. It uses an Ubuntu Python job and does not execute the Wolfram semantic suites. That workflow is valuable, but it should not be mistaken for a native mathematical regression gate.[26]

A recommended layered setup is a fast packaging/style gate for every relevant change, a small native smoke suite for public constructors and arithmetic, family-specific suites selected by modified modules, and a complete native suite before a tagged release. Kernel/build identity, operating system, test selection, and source hashes should be retained as artifacts. Timeout or abort is a failed/unfinished test, not an empty success.

Native execution may require a licensed or otherwise authorized runner. That practical constraint does not justify calling packaging checks mathematical validation. The workflow can state exactly which layer has run and which remains required before release. Cross-platform and more-than-one-supported-kernel checks are especially valuable because the guide already records a version-specific download/loading issue.[2]

10.3 Tests that target semantic invariants

A strong test corpus should combine ordinary golden outputs with properties that survive different algorithms and expression forms. Suggested invariant tests include:

- **Residual and method agreement:** Lagrange, grouped Lagrange, and Newton yield the same complete blocks, and finite-model residuals vanish below the appropriate normalized cutoff.
- **Precision monotonicity:** truncation never creates information; successful refinement meets its requested precision; arithmetic never assumes the remainder is differentiable or nonzero without evidence.

- **Branch and representation equivalence:** direct and nested observables enforce the same real-domain requirements; equivalent source/target shifts, callable forms, and coordinate representations retain the same branch meaning.

Additional generated tests should deliberately target cancellation: several indices at the same weight, disappearance of the first omitted block, exact termination, and subtraction of objects with independently unknown errors. Random coefficients alone will rarely generate these important boundaries. Parameterized tests should vary the sign of leading coefficients, finite versus infinite endpoints, logarithmic degree, and negative observable powers.

Resource tests should assert behavior before allocation. A dense-export planner can be tested with a denominator of 10^{100} without allocating any list. A combinatorial-support counter can prove that a method's planned work exceeds budget before enumerating it. Tests should also distinguish optional simplification failure from mathematical failure and ensure that interrupted refinement leaves the previous object usable.

The included `AuditRegressions.wlt` contains eleven native regression proposals, including direct/nested branch consistency, dense-export refusal, the negative-power refinement target, offset normalization, and several baseline mathematical coefficients. They were not executed in this audit. After applying only the three supplied surgical edits, F04 is still expected to require a separate implementation change; a completely green audit suite is not claimed.

10.4 Reproducible single-file distribution

The generated standalone file and its freshness checker are good distribution choices. They allow simple loading without requiring users to arrange companion modules, while retaining canonical modular sources for development. The guide also explains fixed-commit loading and offline use.[2, 25, 26]

For reproducibility, the first recommended installation path should favor an immutable release or pinned commit, not mutable `main`. This is not an accusation about the repository: loading any remote source executes whatever that source currently contains. A release artifact should carry a manifest linking the standalone file, modular sources, package version, and native validation results.

After a source patch, regenerate the standalone file and rerun the freshness checker. Editing both independently invites drift. The supplied patch generator changes only the canonical modular entry file after explicit authorization; it does not silently regenerate or execute repository scripts. The accompanying instructions identify the separate regeneration and validation steps.

11 A concrete improvement architecture

11.1 One shared semantic kernel, several coefficient algebras

The central refactoring should be a small internal protocol rather than one monolithic universal transseries implementation. A *chart* records variable, endpoint, direction, positive coordinate, source/target substitutions, and domain. A *coefficient algebra* supplies arithmetic, exact equality, derivation, and any growth or dominance metadata. A *support order* supplies exact comparison, finite truncation, and complete collision aggregation. A *precision contract* records what is known about omitted terms and their derivatives.

Constructors and adapters can then produce typed results using those protocols. Ordinary power-log, finite Fourier, inverse-logarithmic polynomial, and flat-sector forms need not share an

identical physical storage layout. They do need to share rules for reporting precision, branch prerequisites, resource use, and operation capabilities.

This design addresses the present findings directly. F02 is prevented when every power operation asks the same domain-aware primitive. F04 is prevented when every refinement request has a checked achieved-precision postcondition. F01 is isolated in an optional conversion service. F03 is avoidable when the inversion algorithm can request support information without being forced into the direct multi-index representation.

11.2 Backward precision propagation

For ordinary jets, the following local rules form a useful initial precision planner. Let the finite leading approximations have valuations α and β and let h be the requested output exponent. These rules assume required nonvanishing and real-branch conditions are already proved.

Operation	Sufficient input exponent requirement
Addition $A + B$	Each input at least h , unless a stronger shared-error dependency is explicitly tracked.
Multiplication AB	$P_A \geq h - \beta$ and $P_B \geq h - \alpha$, with product-error terms included.
Power A^r	$P_A \geq h - (r - 1)\alpha$ for fixed nonzero r .
Logarithm $\log A$	$P_A \geq h + \alpha$.
Derivative in u	Input order at least $h + 1$ plus a valid matching derivative remainder contract.
Exponential e^A	Control absolute error in the phase; use its exact carrier and relative precision, not a bare absolute exponent rule.

These are planning rules, not a replacement for exact post-operation precision calculation. Logarithmic degrees matter at equal power exponents. A changing prefactor or nonlinear coordinate can alter the derivative loss. Cancellation can improve actual precision, but the planner should not rely on accidental cancellation before it is established.

Composition needs a corresponding derivative-based transport calculation. If A is expanded around an endpoint reached by B , the inner error is amplified by the valuation of the derivative of the outer approximation. The outer source remainder must also be composed into the inner coordinate. A fixed guard of two or three orders cannot uniformly cover singular outer observables. An adaptive verification loop remains useful even after analytic demand rules are implemented.

11.3 Independent evidence objects

Separate evidence for coefficient correctness, asymptotic validity, source branch, and numerical certification. A useful result can have a verified finite-model residual and an unproved global branch extension; another can have a rigorous local root interval but only a low-order symbolic expansion. One boolean called "Verified" cannot express these combinations.

A proposed record could include

```
"Evidence" -> <|
  "FiniteModelResidual" -> residualWitness,
  "AsymptoticRemainder" -> remainderContract,
  "SourceBranch" -> branchWitness,
  "NumericalComparison" -> Missing["NotRequested"],
  "IntervalCertificate" -> Missing["NotRequested"]
|>
```

Each witness should identify its assumptions, input hashes or model signature, exact scope, and generation method. Existing detailed result fields can be migrated into such a structure incrementally rather than discarded.

11.4 A strategy planner with observable decisions

A proposed **Method** -> **Automatic** planner should inspect the model rather than defaulting silently to one expensive path. Useful features include number of generators, rational-lattice rank, anticipated index count, expected support collision density, logarithmic coefficient degrees, target cutoff, requested block count, and availability of a reusable state.

Its decision should be returned in metadata: selected method, rejected alternatives, estimated work, and actual counters. Users debugging internals need to know whether an apparent Newton request still caused direct Lagrange frontier evaluation. This also makes performance regressions testable without relying exclusively on noisy wall-clock measurements.

12 Development priorities and acceptance milestones

12.1 Milestone A: repair and make failure honest

First address F01, F02, F04, and F05. The first, second, and fifth have small proposed edits in this bundle; F04 requires at least an achieved-precision check and preferably an adaptive or backward planner. Acceptance is a native regression run showing branch-path consistency, no unbounded dense conversion, correct offset diagnostics, and no successful under-refinement.

At the same time, standardize resource failures with requested/consumed budgets and a best valid result where appropriate. Preserve all current conservative behavior: do not make branch failures disappear by accepting complex results in a real interface, and do not make refinement appear successful by relabeling a weak remainder.

12.2 Milestone B: remove method-independent combinatorial bottlenecks

Decouple Newton and grouped Lagrange from full direct-index construction. Introduce support-aware planning and optional frontier sharpening. Use the exact polynomial family in F03 as an acceptance fixture, with structural counters proving that the hidden enumeration disappeared. Test irrational-gap and resonant rational-gap examples to ensure the optimization did not replace exact support semantics with a lattice-only shortcut.

Bound logarithm normalization and optional conversion work. Instrument polynomial power caches, coefficient expression size, simplification/proof time, and native adapter time. Performance improvements should be demonstrated on public operations as well as helpers.

12.3 Milestone C: stabilize the public model

Add chart, precision, capability, and schema metadata, and a migration path for saved objects. Resolve ambient assumptions once at the public boundary. Introduce explicit observables while preserving the old "**Power**" convention through a documented compatibility layer. Keep compact syntax, but make its desugared meaning inspectable.

This milestone should include a user-facing diagnostic command that explains an object's scale, remainder contract, supported operations, and evidence. The goal is not another layer of prose

documentation alone; it is machine-readable semantics that downstream programs can rely on.

12.4 Milestone D: extend the most economical mathematical cases

Primitive commensurate flat rates are a small, well-delimited extension with a complete exact algorithm. A reusable inverse-logarithmic coefficient algebra would unlock more uniform Gamma/Barnes operations. Fourier term goals and refinement can reuse the existing complete-weight machinery once its method coupling is removed.

A public ordered-series integration operation is another natural addition. It would need special treatment of exponent -1 , logarithmic resonances, constants of integration, endpoint domains, and integrated remainder assumptions. It should be implemented as an operation with its own contract, not by applying ordinary `Integrate` to `Normal[s]` and attaching an unexamined order term.

12.5 Milestone E: expand rigorous numerical evidence

Develop a small independently checkable certificate format for the current elementary evaluator. Then extend to logarithmic Gamma and Barnes equations with proved constant and special-function bounds. Use interval conditioning and monotonicity witnesses to connect numerical approximations with guaranteed error.

A typed Lean formalization of the power-log remainder algebra or the rational interval checker would be a useful optional research direction. The tractable first targets are local: directed rounding lemmas, elementary Taylor tail inequalities, the residual-bracket theorem, and finite coefficient identities. They do not require formalizing every native symbolic simplification or special-function adapter at once.

12.6 Longer-range work that needs a new theory boundary

Complex-sector branches and Stokes transitions, infinite or irrationally independent exponential supports, uniform parameter asymptotics near changing dominant balances, and multivariable implicit systems are not small parser extensions. They require explicit domains, ordering or summability hypotheses, and new remainder semantics. The project can develop these incrementally, but should keep them separate from a near-term promise of arbitrary transseries.

An especially useful intermediate direction is *conditional parameter regions*: instead of rejecting every undecidable sign or exponent comparison, return a finite set of proved parameter cases when a native decision procedure can establish them. Such case splitting must preserve a chart and precision contract per branch. The alternative “unknown within budget” outcome must remain available.

13 Executed independent checks and their interpretation

13.1 Exact checks

The independent script records 22 checks in total. They include the dense-export length, weighted multi-index counts, primitive rational sector lattices, generalized Lagrange coefficient composition, finite Gamma and Barnes phase reversion, elementary interval-tail checks, and high-precision inverse comparisons. All 22 passed in the recorded run. Eleven further unit tests checked the

audit’s patch/planning tooling, including rejection of an unrecognized source file and rejection of duplicate patch application.

These checks are independent in a specific sense: SymPy polynomial manipulation and exact integer/Fraction arithmetic were used rather than evaluating the Wolfram package. The Gamma and Barnes oracle equations deliberately test the same mathematical finite models; agreement validates their algebraic reversion, not the correctness of every source dispatcher that eventually reaches those models.

For the weighted support in F03, the independently computed counts are:

Exclusive weight H	20	24	28	30	32	36
Indices, gaps 1:30	2,087	5,763	14,742	23,025	35,470	81,130

The counts beyond 30 still use the same 30 generators. They are not the unrestricted partition cumulative counts once weights exceed the largest supplied generator.

13.2 Irrational-power inverse check

For $f(x) = x + x^{\sqrt{2}}$, the independent generalized Lagrange approximation retained index depths $n = 0, \dots, 5$. Its first unretained power is $1 + 6(\sqrt{2} - 1)$. High-precision roots were compared with this approximation:

Target y	Absolute error	Error/ $y^{1+6(\sqrt{2}-1)}$
10^{-4}	$1.5453164625 \times 10^{-13}$	13.4940690512
10^{-8}	$1.8441485495 \times 10^{-27}$	14.0619935771
10^{-12}	$2.1138478515 \times 10^{-41}$	14.0750587314

The stabilization is consistent with the next nonzero coefficient controlling the remainder. It is numerical evidence at three points, not a proof of a uniform error bound or a native-package benchmark.

13.3 Gamma and Barnes inverse check

The independent code evaluated approximations with four unit-correction orders around chosen exact core values. It solved the corresponding logarithmic equations at high precision and obtained:

Core value X	Gamma inverse absolute error	Barnes inverse absolute error
20	$2.0627485309 \times 10^{-9}$	$1.1067660707 \times 10^{-7}$
50	$5.3148322814 \times 10^{-11}$	$3.0626798521 \times 10^{-10}$
100	$2.7815160164 \times 10^{-12}$	$2.0061398108 \times 10^{-12}$

These observations support the scaling and signs of the checked coefficient equations. The absolute errors for different families are not normalized by the same remainder scale, so the table should not be read as an accuracy ranking of Gamma versus Barnes algorithms.

The elementary interval checks in the same script compare rational Taylor-tail enclosures with high-precision values. They are useful sanity checks of the selected formulas. They are not a machine-checked proof of the upstream certificate evaluator and do not exercise its exact directed rounding implementation.

13.4 Native results still to obtain

The native regression suite, native comparison collector, and proposed Wolfram helper functions have not been run here. The most important pending evidence is the actual F02 and F04 reproduction on the pinned kernel version, followed by the same cases on the corrected implementation. The full upstream suite and modular/standalone parity should then be run against the resulting source hashes.

A failed proposed regression may reveal a test fixture issue, a different evaluation path, or a genuine defect. The native runner records actual outputs and messages so that those possibilities can be distinguished. This article’s classification should be updated from “source-traced” to “natively reproduced” only when that evidence exists.

14 Conclusion

Asymptotic has a coherent mathematical center and several unusually careful engineering choices: exact generalized weights, complete collision blocks, explicit derivative assumptions, finite-model residual checks, immutable refinement state, conservative composite errors, and exact interval certificates with a clearly limited scope. The independent checks found no contradiction in the reviewed Euler coefficient, remainder transport, or finite Gamma/Barnes reversion mechanisms. That is positive evidence about those mechanisms, not a proof of the entire package.[3, 7, 21, 6, 11]

The weaknesses cluster around the boundaries between those mechanisms. An optional native representation can allocate far more than the mathematical result; a generic recursive evaluator can bypass a public branch check; a refinement wrapper can confuse the requested cutoff with the achieved precision; and a method selector can leave an expensive representation requirement in place before the chosen algorithm starts.

Fixing those boundaries will improve more than the individual examples. A common domain-aware operation kernel, a checked precision postcondition, a separate conversion service, and a method-independent support planner would make future features safer to add. The strongest roadmap is therefore not simply a longer list of supported special functions. It is a smaller number of shared invariants, made explicit and enforced across every scale.

A Artifact guide and reproduction commands

The archive is a review bundle, not a full clone of the repository. It includes the article, independent executable checks, native reproduction programs, a guarded patch generator, and evidence metadata. Upstream code remains available through the pinned repository links in the references.

```

article/asymptotic-audit.tex
article/asymptotic-audit.pdf
code/audit_oracles.py
code/apply_proposed_fixes.py
code/test_audit_tools.py
wolfram/AuditRegressions.wlt
wolfram/RunAudit.wls
wolfram/CompareBuiltin.wls
wolfram/ProposedHelpers.wl
results/oracles.json
results/tool-tests.txt
evidence/snapshot.json
evidence/findings.json

```

```
evidence/source-map.json
README.md
LICENSE
MANIFEST.sha256
```

From the extracted bundle directory, rerun the independent checks with:

```
python -m pip install sympy==1.14.0 mpmath==1.3.0
python code/audit_oracles.py --output results/oracles.json
python -m unittest discover -s code -p test_audit_tools.py -v
```

The scripts target Python 3.9 or later. The recorded execution environment was Python 3.13.5. Installation is shown for reproducibility; no package installation is performed automatically by the scripts.

The patch generator defaults to a dry run:

```
python code/apply_proposed_fixes.py /path/to/Asymptotic \
--output proposed_fixes.patch
```

It verifies the canonical file’s Git blob hash, checks every replacement anchor count, and refuses a different source version. Passing `-apply` explicitly requests modification and creates a non-overwriting backup first. The patch addresses only F01, F02, and F05. It does not implement the Newton planner, derived-refinement redesign, or a complete resource policy.

After reviewing and applying a patch, regenerate the repository’s standalone source using its existing builder, then run the native regressions. A PowerShell example is:

```
$env:ASYMPTOTIC_REPO = 'C:\src\Asymptotic'
python C:\src\Asymptotic\validation\build_standalone.py
wolframscript -file .\wolfram\RunAudit.wls
wolframscript -file .\wolfram\CompareBuiltins.wls
```

The comparison collector is not a pass/fail capability benchmark. It retains inputs, outputs, messages, and timeouts for analysis. Its sequential timings are not fresh-kernel measurements for each individual case. The audit regression suite does not replace the repository’s complete native test suite.

To rebuild the article, run `latexmk -pdf asymptotic-audit.tex` from the `article` directory, or run `pdflatex` repeatedly until cross-references settle. No external images, font files, bibliography database, or network access are required by the document.

B Module coverage and architectural map

The following table makes review scope explicit. “Substantial” means the main code path and relevant helpers were read, not that every possible evaluation branch was exercised. “Interface” means documentation, loader inventory, and call-site relationships were reviewed, but a complete implementation audit is not claimed. All paths are under `AsymptoticInverse/Kernel/` unless otherwise stated.

Module	Review depth	Role or emphasis
<code>AsymptoticInverse.wl</code>	Substantial	Main model, jets, power/log operations, constructor, export, residual diagnostics.

Module	Review depth	Role or emphasis
SeriesOperations.wl	Full source	Explicit operations, derivative contracts, refinement replay; F02/F04.
SeriesArithmetic.wl	Selected source	Ordinary operator integration and normalization paths.
SeriesEnvelopeArithmetic.wl	Substantial	Separate error envelopes, approach compatibility, resource estimates.
IncrementalInverse.wl	Full source	Cached coefficients, complete weight layers, Newton doubling, grouped Lagrange.
RefinementState.wl	Substantial	State signatures, prefix checks, reuse, frontier and reconstruction.
RefinementRequests.wl	Full source	Additional-block and numerical accuracy request semantics.
ExactTermination.wl	Full source	Bounded exact-source verification of termination.
InverseCertificates.wl	Full source	Rational enclosure arithmetic, domain/derivative checks, adaptive certification.
NumericalInverseChecks.wl	Full source	Exact target substitution, numerical root/observable comparison, source-domain checks.
NativeSpecialFunctions.wl	Full source	Native tree import, real projection, carriers, error transport.
InverseFunctionExpressions.wl	Substantial	Public dispatch, nested inverse expressions, branch provenance.
InverseFunctionBranches.wl	Interface	Branch discovery and selection service.
InverseFunctionFamilies.wl	Interface	Recognized inverse-function families.
InverseFunctionSyntax.wl	Interface	Callable and inverse-operator syntax.
GammaInverse.wl	Substantial	Lambert core, ordered coefficients, bounds, parameter and cutoff validation.
BarnesInverse.wl	Full source	Barnes coefficient equation and sufficient increasing branch.
GammaInverseOperations.wl	Interface	Family-specific operations.
GammaInverseChecks.wl	Interface	Family-specific inverse evidence.
BarnesInverseChecks.wl	Interface	Barnes residual/check adapter.
GammaForward.wl	Interface	Forward Gamma normalization and asymptotics.
BarnesForward.wl	Interface	Forward Barnes normalization and asymptotics.
ExponentialForward.wl	Interface	Factored elementary exponential forward results.
FourierCoefficients.wl	Substantial	Exact Fourier algebra, source-weight multiplication, constructor admission.
FlatSectors.wl	Full source	Exact monomial core, rates, sector coefficients and majorant; F07.
FlatSectorOperations.wl	Interface	Flat-sector arithmetic and derivative operations.
CorePerturbation.wl	Interface	Perturbations around an exact inverse core.
ExponentialCorePerturbation.wl	Interface	Exact exponential-core perturbation route.
LambertInverse.wl	Interface	Recognized Lambert inversion charts.
LogarithmicScales.wl	Interface	Leading logarithmic and hierarchical coefficient scales.
ReciprocalLogOperations.wl	Interface	Reciprocal-logarithmic composition/calculus.
CoordinateInverse.wl	Interface	Coordinate-specific inverse reconstruction.
SourceCoordinates.wl	Interface	Source coordinate recognition and transformations.
SpecialFunctionAdapters.wl	Interface	Adapter-specific inverse models.

Module	Review depth	Role or emphasis
<code>SpecialFunctionIdentities.wl</code>	Interface	Identity-based normalization.
<code>SpecialFunctionRealDomain.wl</code>	Interface	Proved sufficient real domains.
<code>ParameterizedSpecialFunctions.wl</code>	Interface	Parameter-dependent special-function normalization.
<code>DirichletSpecialFunctions.wl</code>	Interface	Defining-sum special-function asymptotics.

The guide, current validation README, focused runner and selection, standalone workflow, master article, and the article’s remainder/transport sections supplied additional context. No complete per-line review is claimed for every `.wlt` file or every archived research report. Native coverage of all entries in this table is zero in this audit environment; the executed evidence is the independent Python work described separately.

C Suggested regression families beyond the included cases

A small set of systematic families can expose more than many unrelated examples. The following are proposed additions, not claims that current behavior fails all of them.

Coordinate and sign symmetry. Begin with a positive finite-endpoint example, then transform it by a real source translation, source reflection, positive/negative target scale, and reciprocal source chart. Check that reconstruction, remainder variable, source domain, numerical checks, and inverse powers all transform consistently. Retain separate tests for the special $r = 1$ displacement convention.

Precision amplification. Apply powers $-1, -2, -10$, logarithms, and compositions with poles to a source of known valuation. Compare the final achieved precision with the backward planner. Include zero leading-block cancellation, a pure remainder, exact zero, and a coefficient whose sign needs assumptions. Refinement should either meet the goal or return an explicit obstruction.

Collision stress. Use gaps $1, 2, \dots, m$ with coefficient choices that force entire weights to cancel, and compare direct, grouped, and Newton results. Add algebraic but structurally different equal weights to test canonical equality. Frequency collisions should be tested independently from source-weight collisions.

Boundary evidence. Test certificates whose derivative interval just includes zero, whose center residual interval includes zero, whose residual bracket touches an endpoint, and whose source condition fails at only part of the closed interval. Test a fixed center with an unattainable requested accuracy and a relative request near a zero root. These cases target proof obligations, not just floating-point agreement.

Special-function import. Use multiple carriers, products with cancelling exponential growth, bounded real oscillations, complex-looking native expressions for a real source, and a native expression without an explicit order term. Check that exact equality is required before treating the last case as exact and that missing logarithmic tail degree is handled conservatively.

Distribution and saved results. Run the same public examples through the modular package and regenerated standalone file, compare normalized metadata as well as expressions, and reload saved schema versions. Exercise invalid option names, malformed associations, assigned source symbols, empty reports, missing files, interrupted calls, and reproducible failure output.

References

- [1] Vladimir Reshetnikov, *Asymptotic*, audited repository snapshot, 9 September 2026. Commit 07a9781212beb2eeb9ff16aa625b50ac27974078. README.md. All repository references below use this immutable commit unless stated otherwise.
- [2] *AsymptoticInverse User Guide*. AsymptoticInverse/Documentation/UserGuide.md. Public APIs, version requirements, cutoff conventions, branch rules, loading and migration.
- [3] *Main kernel implementation*. AsymptoticInverse/Kernel/AsymptoticInverse.wl. See especially `fwdPower`, `logCanon`, `construct`, `makeSeriesData`, `makeInverseSeriesData`, and `InverseResidual`.
- [4] *Explicit series operations*. AsymptoticInverse/Kernel/SeriesOperations.wl. See `seriesPower`, `seriesJetApply`, `seriesDerivative`, `SeriesRefine`, and `seriesRefinementResult`.
- [5] *Ordinary arithmetic integration*. AsymptoticInverse/Kernel/SeriesArithmetic.wl.
- [6] *Composite asymptotic envelope arithmetic*. AsymptoticInverse/Kernel/SeriesEnvelopeArithmetic.wl.
- [7] *Incremental and grouped inverse algorithms*. AsymptoticInverse/Kernel/IncrementalInverse.wl.
- [8] *Retained refinement state*. AsymptoticInverse/Kernel/RefinementState.wl.
- [9] *Structured refinement requests*. AsymptoticInverse/Kernel/RefinementRequests.wl.
- [10] *Exact termination checks*. AsymptoticInverse/Kernel/ExactTermination.wl.
- [11] *Exact inverse certificates*. AsymptoticInverse/Kernel/InverseCertificates.wl.
- [12] *Numerical inverse checks*. AsymptoticInverse/Kernel/NumericalInverseChecks.wl.
- [13] *Native special-function expansion importer*. AsymptoticInverse/Kernel/NativeSpecialFunctions.wl.
- [14] *Applied inverse expressions and dispatch*. AsymptoticInverse/Kernel/InverseFunctionExpressions.wl.
- [15] *Gamma/Barnes inverse construction*. AsymptoticInverse/Kernel/GammaInverse.wl.
- [16] *Barnes inverse coefficients and branch*. AsymptoticInverse/Kernel/BarnesInverse.wl.
- [17] *Finite Fourier coefficient algebra*. AsymptoticInverse/Kernel/FourierCoefficients.wl.
- [18] *Finite flat-sector inversion*. AsymptoticInverse/Kernel/FlatSectors.wl.
- [19] *Exact-core perturbation interface and implementation*. AsymptoticInverse/Kernel/CorePerturbation.wl; see also the guide for public contracts.
- [20] *Remainders after a cutoff*, repository mathematical article. `article/sections/08-remainders.tex`.
- [21] *Transport of remainders and finite smoothness*, repository mathematical article. `article/sections/09-transport.tex`.
- [22] *Validation notes and recorded test/benchmark scope*. `validation/README.md`. Repository-reported evidence, not independently rerun here.
- [23] *Focused native validation runner*. `validation/FocusedTests.wl`.
- [24] *GeneralizedSeries refactoring test selection*. `validation/CheckGeneralizedSeries.wl`.
- [25] *Standalone package distribution*. AsymptoticInverse.wl; builder at `validation/build_standalone.py`.

- [26] *Standalone freshness CI workflow*. [.github/workflows/standalone.yml](https://github.com/workflows/standalone.yml).
- [27] Wolfram Research, *Series*, Wolfram Language documentation. <https://reference.wolfram.com/language/ref/Series.html>. Accessed 9 September 2026.
- [28] Wolfram Research, *SeriesData*. <https://reference.wolfram.com/language/ref/SeriesData.html>. Accessed 9 September 2026.
- [29] Wolfram Research, *Asymptotic*. <https://reference.wolfram.com/language/ref/Asymptotic.html>. Accessed 9 September 2026.
- [30] Wolfram Research, *AsymptoticSolve*. <https://reference.wolfram.com/language/ref/AsymptoticSolve.html>. Accessed 9 September 2026.
- [31] Wolfram Research, *InverseSeries*. <https://reference.wolfram.com/language/ref/InverseSeries.html>. Accessed 9 September 2026.
- [32] Wolfram Research, *ComposeSeries*. <https://reference.wolfram.com/language/ref/ComposeSeries.html>. Accessed 9 September 2026.
- [33] Wolfram Research, *AsymptoticIntegrate*. <https://reference.wolfram.com/language/ref/AsymptoticIntegrate.html>. Accessed 9 September 2026.
- [34] Wolfram Research, *AsymptoticDSolveValue*. <https://reference.wolfram.com/language/ref/AsymptoticDSolveValue.html>. Accessed 9 September 2026.
- [35] Wolfram Research, *Assuming*. <https://reference.wolfram.com/language/ref/Assuming.html>. Accessed 9 September 2026.
- [36] Wolfram Research, *FactorInteger*. <https://reference.wolfram.com/language/ref/FactorInteger.html>. Accessed 9 September 2026.
- [37] NIST Digital Library of Mathematical Functions, §5.11, *Asymptotic Expansions of the Gamma Function*, especially the error-bound discussion. <https://dlmf.nist.gov/5.11>. Accessed 9 September 2026.
- [38] NIST Digital Library of Mathematical Functions, §5.17, *Barnes G-Function (Double Gamma Function)*. <https://dlmf.nist.gov/5.17>. Accessed 9 September 2026.